

Gráficos sobre la vida real

Contenido

- 17.1. Node2Vec
- 17.2. Ejercicios

Durante el curso hemos visto muchas bibliotecas y herramientas que se pueden usar para crear gráficos. Además, hemos visto cómo navegar por los datos y cómo crear un gráfico a partir de ellos. Finalmente, en las últimas semanas hemos visto cómo explorar el gráfico y cómo extraer información de él.

En esta sección, aplicaremos todos los conocimientos adquiridos a un caso real. Utilizaremos los datos de la biblioteca `PytorchGeometric` para crear un gráfico y explorarlo. No te preocupes si no estás familiarizado con la biblioteca; te guiaremos durante el proceso y te proporcionaremos la información y el código necesarios para realizar las tareas.

Nota

Es importante mencionar que el objetivo de este curso no es enseñar cómo funcionan los diferentes modelos de aprendizaje automático o aprendizaje profundo, sino mostrar cómo crear un grafo a partir de un conjunto de datos y cómo explorarlo. Si le interesa obtener más información sobre los modelos, le recomendamos consultar la documentación oficial de la biblioteca o del modelo que le interese.

17.1. Node2Vec

En las clases anteriores, hemos visto cómo realizar un paseo aleatorio en un grafo. Un paseo aleatorio es un proceso estocástico que describe una trayectoria compuesta por una secuencia de pasos aleatorios en un grafo. El paseo aleatorio comienza en un nodo dado del grafo y luego se mueve a un nodo vecino con cierta probabilidad. El paseo aleatorio continúa de esta manera hasta llegar a un nodo terminal. El paseo aleatorio se puede utilizar para explorar el grafo y encontrar patrones interesantes en él.

Uno de los algoritmos que se pueden usar para realizar un recorrido aleatorio en un grafo es el algoritmo Node2Vec. Este algoritmo de aprendizaje de representaciones se utiliza para aprender las incrustaciones de los nodos en un grafo. Las incrustaciones de los nodos se aprenden de tal manera que los nodos cercanos en el grafo también estén cercanos en el espacio de incrustaciones. El algoritmo Node2Vec se basa en la idea de usar un recorrido

[Saltar al contenido principal](#)

Pero antes de poder utilizar el algoritmo Node2Vec para aprender las incrustaciones de los nodos en un grafo, necesitamos comprender el significado de las incrustaciones y cuán poderosas son.

17.1.1. Incrustaciones y word2vec

Las incrustaciones de los nodos en un grafo son las representaciones de baja dimensión de los nodos en el grafo. Estas representaciones se aprenden de tal manera que los nodos que están cerca en el grafo también están cerca en el espacio de incrustación. Las incrustaciones de los nodos se pueden utilizar para realizar diversas tareas en el grafo, como clasificación, agrupamiento y predicción de enlaces. Las incrustaciones en grafos son similares a las incrustaciones en texto, donde las palabras que están cerca en el texto también están cerca en el espacio de incrustación. Por eso, el algoritmo Node2Vec se basa en el algoritmo Word2Vec. Word2Vec es un algoritmo de aprendizaje de representaciones que se utiliza para aprender las incrustaciones de las palabras en un texto. Las incrustaciones de las palabras se aprenden de tal manera que las palabras que están cerca en el texto también están cerca en el espacio de incrustación.

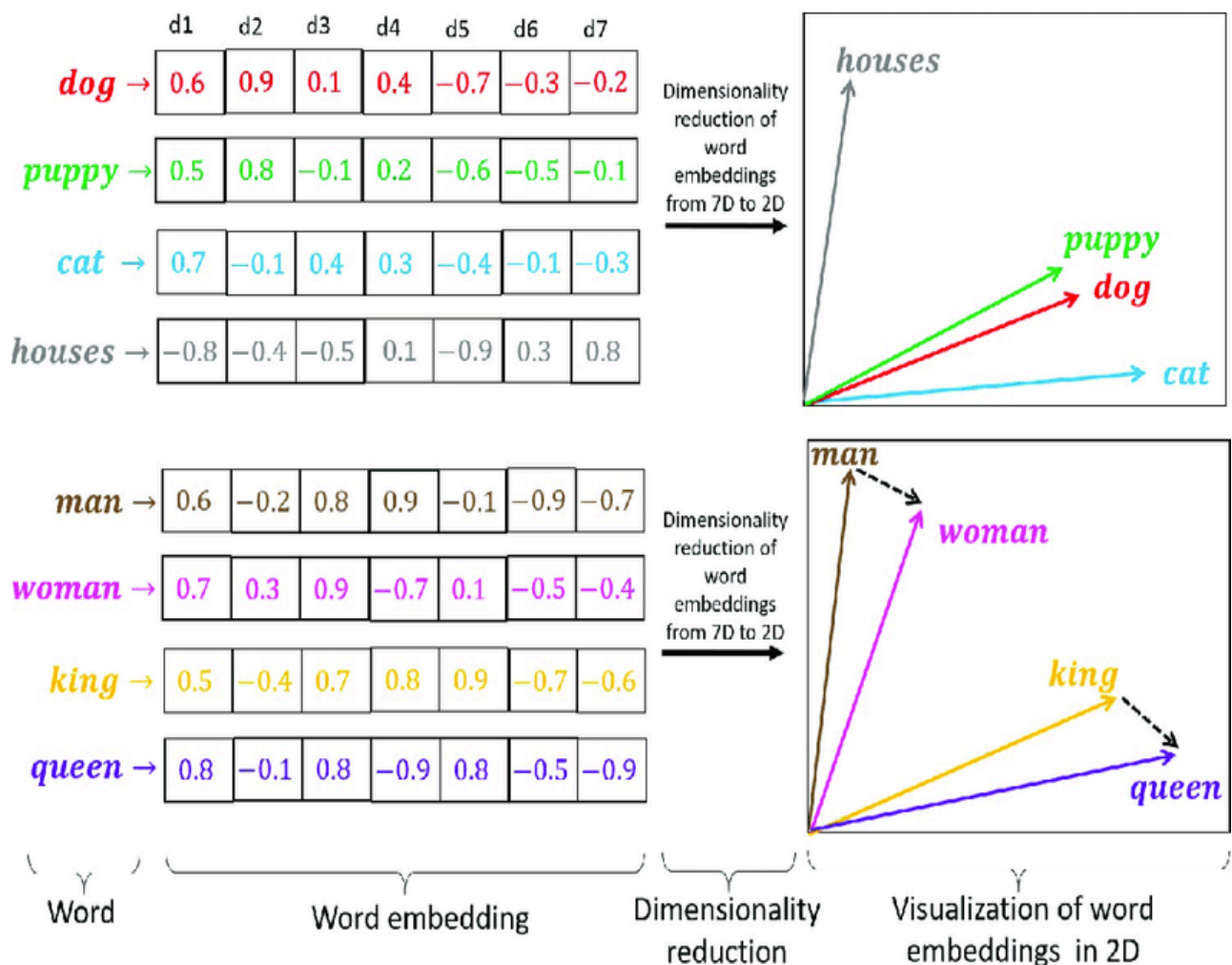


Figura 17.1 Idea de incrustaciones

Las incrustaciones son útiles en el campo del aprendizaje automático y el aprendizaje

[Saltar al contenido principal](#)

Por ejemplo, en visión artificial, podemos usar incrustaciones para representar imágenes de millones de píxeles de forma más compacta y significativa. Lo mismo ocurre con las incrustaciones de los nodos en un grafo: podemos representarlos de forma más compacta y significativa. Pero lo más importante de las incrustaciones es que capturan toda la información en pocas dimensiones.

17.1.2. Algoritmo Node2Vec

Siguiendo la idea de Word2Vec, el algoritmo Node2Vec es un algoritmo de aprendizaje de representaciones que se utiliza para aprender las incrustaciones de los nodos en un grafo. Las incrustaciones de los nodos se aprenden de tal manera que los nodos que están cerca en el grafo también están cerca en el espacio de incrustaciones.

A continuación se ofrece una breve explicación de cómo funciona el algoritmo Node2Vec:

1. **Generación de recorridos aleatorios** : El primer paso del algoritmo Node2Vec consiste en generar recorridos aleatorios en el grafo. Estos recorridos se generan partiendo de un nodo dado y moviéndose a un nodo vecino con cierta probabilidad. El recorrido aleatorio continúa de esta manera hasta llegar a un nodo terminal.
2. **Aprendizaje de las incrustaciones** : El segundo paso del algoritmo Node2Vec consiste en aprender las incrustaciones de los nodos en el grafo. Estas incrustaciones se aprenden de forma que los nodos cercanos en el grafo también lo estén en el espacio de incrustaciones. El aprendizaje de las incrustaciones se realiza mediante el modelo Skip-Gram, una variante del modelo Word2Vec.

17.1.2.1. Fragmento de código

Aquí hay un fragmento de código que muestra cómo usar el algoritmo Node2Vec para aprender las incrustaciones de los nodos en un grafo:

```
import numpy as np
import networkx as nx
from gensim.models import Word2Vec

# Función para realizar una caminata aleatoria en el grafo
def node2vec_walk(G, start_node, walk_length):
    walk = [start_node]
    while len(walk) < walk_length:
        cur_node = walk[-1]
        neighbors = list(G.neighbors(cur_node))
        if len(neighbors) > 0:
            walk.append(np.random.choice(neighbors)) # Selecciona un vecino
        else:
            break
    return walk

# Función para generar múltiples caminatas aleatorias desde cada nodo del gr
def generate_walks(G, num_walks, walk_length):
    walks = []
```

[Saltar al contenido principal](#)

```

np.random.shuffle(nodes) # Mezcla los nodos para añadir aleatoriedad
for node in nodes:
    walks.append(node2vec_walk(G, node, walk_length)) # Genera una
return walks

# Función principal de Node2Vec
def node2vec(G, dimensions, num_walks, walk_length):
    walks = generate_walks(G, num_walks, walk_length) # Genera todas las ca
    walks = [list(map(str, walk)) for walk in walks] # Convierte los nodos
    # walks: Lista de caminatas aleatorias
    # vector_size=dimensions: Tamaño del vector de embeddings
    # window=5: Tamaño de la ventana de contexto, en nodos sería 5 nodos a l
    # workers=2: Número de hilos para entrenar el modelo
    model = Word2Vec(walks, vector_size=dimensions, window=5, workers=2) #
    return model

```

17.1.3. Node2Vec en acción

Ahora que hemos visto cómo funciona el algoritmo Node2Vec, veamos cómo usarlo en la práctica generando las incrustaciones de los nodos en un grafo.

```

import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
G = nx.stochastic_block_model([20, 20, 20], [
    [0.3, 0.01, 0.01],
    [0.01, 0.3, 0.01],
    [0.01, 0.01, 0.3]
], seed=42)

# Mostar el grafo
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 8))
plt.title("SBM Graph")
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True, node_color='skyblue', edge_color='black')
plt.show()

model = node2vec(G, dimensions=16, num_walks=10, walk_length=80) # Entrena

# Obtiene el embedding de un nodo específico (por ejemplo, nodo 0)
node_id = 0
embedding = model.wv[str(node_id)]
print(f"Embedding for node {node_id}: {embedding}") # Embedding for node 0:

# Obtiene los embeddings de todos los nodos
embeddings = np.array([model.wv[str(node)] for node in G.nodes()])
print(f"Embeddings shape: {embeddings.shape}") # Embeddings shape: (60, 16)

```

[Saltar al contenido principal](#)

i Nota

La instrucción `model.wv[str(node_id)]` se utiliza para obtener la incrustación de un nodo específico. La instrucción `model.wv[str(node)]` se utiliza para obtener las incrustaciones de todos los nodos en el grafo. `model.wv` es un diccionario que contiene las incrustaciones de los nodos en el grafo. La forma en que obtuvimos este diccionario es entrenando el modelo con la función `node2vec`.

17.1.4. ¿Cómo podemos visualizar las incrustaciones?

Ahora que hemos aprendido las incrustaciones de los nodos en el grafo, podemos visualizarlas en un espacio de baja dimensión utilizando técnicas de reducción de dimensionalidad como PCA o t-SNE. Estas técnicas nos permiten reducir la dimensionalidad de las incrustaciones de un espacio de alta dimensión a uno de baja dimensión, de modo que podamos visualizarlas en un espacio 2D o 3D. Si no estás familiarizado con estas técnicas, no te preocupes, te proporcionaremos videotutoriales y documentación para que aprendas más sobre ellas.

Aquí hay un fragmento de código que muestra cómo visualizar las incrustaciones de los nodos en un grafo usando PCA:

```
from sklearn.decomposition import PCA

# Reduce the dimensionality of the embeddings using PCA
pca = PCA(n_components=2)
embeddings_pca = pca.fit_transform(embeddings)

# Plot the embeddings in a 2D space
plt.figure(figsize=(8, 8))
plt.scatter(embeddings_pca[:, 0], embeddings_pca[:, 1], c='skyblue')
plt.title("Node2Vec embeddings (PCA)")
plt.show()
```

[Saltar al contenido principal](#)

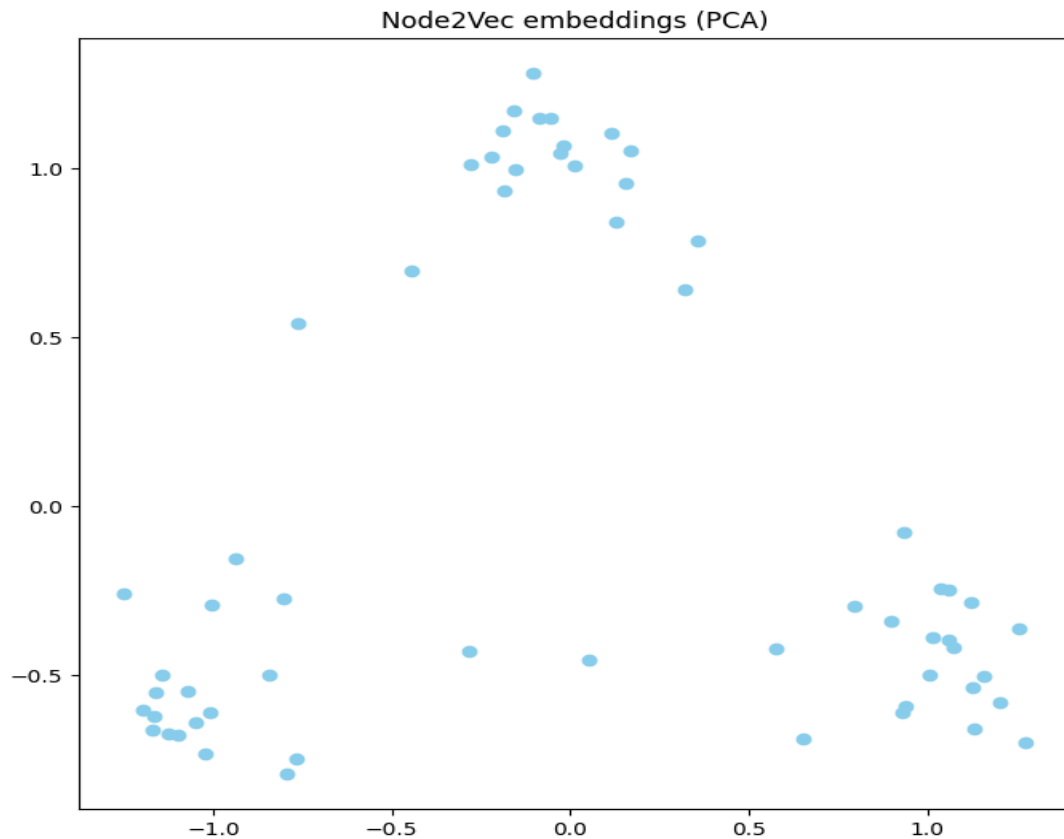


Figura 17.2 Idea de incrustaciones

17.2. Ejercicios

17.2.1. Ejercicio 1

Como hicimos en las clases anteriores, después de leer la sección anterior, deberías poder crear tu Jupyter Notebook y escribir un informe sobre todo lo que has aprendido en esta sección. Para tener una buena estructura, te recomendamos que sigas exactamente la misma estructura que en este notebook. **IMPORTANTE: Quiero un informe muy detallado, con todas las explicaciones y fragmentos de código que has utilizado para realizar las tareas. Al igual que en las prácticas anteriores donde hicimos un seguimiento de los algoritmos.**

17.2.2. Ejercicio 2

Ahora que has aprendido a usar el algoritmo Node2Vec para aprender las incrustaciones de los nodos en un grafo, quiero que apliques el algoritmo Node2Vec al grafo que generamos en las clases anteriores (4 + 2 Grafos). Luego, visualiza las incrustaciones de los nodos en un espacio de baja dimensión usando PCA o t-SNE. ¿Qué puedes decir sobre las incrustaciones de los nodos en el grafo? ¿Los nodos que están cerca en el grafo también están cerca en el

[Saltar al contenido principal](#)

espacio de incrustaciones? ¿Qué puedes decir sobre la calidad de las incrustaciones? Y si pudieras mejorar la calidad de las incrustaciones, ¿cómo lo harías?

Nota

El objetivo de este ejercicio es mostrar cómo aplicar el algoritmo Node2Vec a un grafo y cómo visualizar las incrustaciones de los nodos en un espacio de baja dimensión. Además, se trata de experimentar con los parámetros del algoritmo Node2Vec y observar cómo afectan a la calidad de las incrustaciones.

Anterior

< [16. Exploración de grafos:
Recorridos aleatorios](#)

Próximo

[18. Gráficos sobre la vida real](#) >